

实验1.2

一、背景

本次实验要求编写一个**重力四子棋（Connect Four）残局求解决策程序**，分别使用 `minimax search` 和 `alpha-beta pruning` 两种方法，在双方都采取最优策略的前提下，为当前局面选择下一步最优落子。

对于博弈问题，考虑极小极大搜索算法。规定：

- `MAX` 代表程序方；
- `MIN` 代表对手方；
- `P` 代表一个棋局状态。

若某个状态更有利于 `MAX`，则该状态的效用值取较大值；若更有利于 `MIN`，则该状态的效用值取较小值；若双方最终都无法获胜，则为平局。

基于这个前提，博弈双方要考虑：如何产生一个最好的动作，使其在“对手也采用最优策略”的情况下，仍然能够得到最优结果。

二、问题描述

本实验采用标准重力四子棋棋盘，大小固定为 `6 x 7`。

- 两名玩家分别使用棋子 `x` 和 `o`；
- 每一步只能选择一列落子；
- 棋子会自动落到该列**最底部的空位置**；
- 若某一方形成连续 `4` 个同色棋子（横、竖、斜线均可），则该方立即获胜；
- 若棋盘已满且无人连成四子，则为平局。

以上规则与标准 Connect Four 问题一致。关于游戏规则定义，可参考：

- Wikipedia: https://en.wikipedia.org/wiki/Connect_Four
- Wolfram MathWorld: <https://mathworld.wolfram.com/Connect-Four.html>

本实验中：

- 假定 `x` 方为 `MAX`；
- 假定 `o` 方为 `MIN`。

为了把重点放在博弈搜索本身，而不是放在评估函数设计上，助教给出的测试数据均为**合法且未结束的残局局面**，并满足：

- 剩余空位数不超过 `8`。

这意味着必做部分可以直接搜索到终局，因此**不要求设计启发式评估函数**。

你需要完成的是：在给定残局下，用 `minimax` 和 `alpha-beta pruning` 分别求解当前局面的最优落子，并比较两者展开节点数的差异。

棋盘定义

输入棋盘用 6 行、每行 7 个字符表示，字符含义如下：

- x：x 方棋子
- o：o 方棋子
- .：空位置

例如：

```
1 | x
2 | .....
3 | .....
4 | .....
5 | ..OX...
6 | ..OX...
7 | .XXOO..
```

第一行的 x 表示当前轮到 x 行棋；下面 6 行表示当前棋盘状态，自上而下给出。

终局效用

对于搜索到的终局状态，可采用如下效用定义：

- x 获胜：+1
- o 获胜：-1
- 平局：0

若当前轮到 x，则根节点视为 MAX 节点；若当前轮到 o，则根节点视为 MIN 节点。

搜索要求

完成 minimax 搜索后，再完成 alpha-beta 剪枝算法。

其中需要注意：

1. 后继动作必须按**从左到右**的列顺序展开，即 0,1,2,3,4,5,6；
2. 若存在多个同样最优的落子，输出**列编号最小**的那个；
3. alpha-beta 的最终结果必须与 minimax 完全一致；
4. 统计两种方法各自展开的节点数。

这里的节点计数规则统一为：

- 每进入一次递归搜索函数并处理一个棋盘状态，就记为 1 个展开节点；
- 根节点外层“依次尝试每个合法动作”的包装逻辑不计入节点数；
- 必做部分默认不使用记忆化搜索或置换表，否则节点数不便统一比较。

输入

详见 input/，共有 10 个测试文件，形式如下：

- 第 1 行：一个字符，表示当前轮到哪一方行棋：
 - x：轮到 x 行棋
 - o：轮到 o 行棋
- 接下来 6 行：每行 7 个字符，描述棋盘状态

举例：

```
1 | x
2 | ....OOX
3 | O...XOX
4 | O.OOXXX
5 | OOXXOXO
6 | XXXOOXO
7 | XOXOXOX
```

输出

结果输出到 `output/` 文件夹下，文件命名为 `output_x.txt`，其中 `x` 表示对应的第 `x` 个输入样例。
例如对于 `input_0.txt`，输出文件名为 `output_0.txt`。

输出形式如下：

- 第一行：一个整数，表示你选择落子的列编号，范围为 0 到 6
- 第二行：当前局面在双方都采取最优策略时的最终结果：
 - X_WIN
 - O_WIN
 - DRAW
- 第三行：两个整数，分别表示：
 - minimax 展开的节点数
 - alpha-beta 展开的节点数

举例：

```
1 | 3
2 | X_WIN
3 | 12873 4121
```

代码框架

代码主体框架已在 `src/main.cpp` 中给出，说明如下：

1. 已给出棋盘读入、合法动作生成、落子生成新状态、连四判断和输出写回等基础代码；
2. 你需要补全完成以下几个部分：
 - minimaxValue

- `alphaBetaValue`
- `solveWithMinimax`
- `solveWithAlphaBeta`

3. 给出的代码框架仅供参考，同学们可以在不改变输入输出格式的前提下自行调整实现；
4. 只需保证最后程序能够对所有输入样例产生正确输出即可。

编译运行示例

```
1 | bash run_all.sh
```

该脚本会自动：

1. 编译 `src/main.cpp`
2. 依次处理 `input/` 下所有 `input_*.txt`
3. 将结果写入 `output/output_*.txt`

三、作业要求

算法实现

1. 阅读并理解极小极大搜索和 `alpha-beta` 剪枝算法的相关理论；
2. 编写 `C/C++` 代码，实现：
 - 合法动作生成
 - 终局判断
 - `minimax` 搜索
 - `alpha-beta pruning`
3. 在相同输入下，分别统计 `minimax` 和 `alpha-beta` 的展开节点数；
4. `alpha-beta` 必须保证与 `minimax` 返回相同的最优动作和最终结果；
5. 本实验更关注你对博弈搜索过程的理解，而不要求额外设计复杂评估函数。

提高内容（非必做）

在完成必做部分后，可进一步尝试：

1. 在 `alpha-beta` 中加入走子排序，比较排序前后节点数变化；
2. 将输入扩展到空位更多的局面，引入深度上限与评估函数；
3. 实现一个命令行下的人机对弈程序。

实验报告撰写

撰写实验报告，至少包括以下内容：

1. 棋盘状态表示方式；
2. 合法动作与终局判断的实现方法；
3. minimax 的主要思路；
4. alpha-beta 的主要思路；
5. 两种方法结果一致性的说明；
6. 展开节点数的比较，以及对剪枝效果的分析。

若你自行实现了提高内容，例如：

- 走子排序；
- 更一般中局上的深度截断搜索；
- 评估函数；
- 人机对弈；

也欢迎在报告中单独说明。